RÉSUMÉ DE COURS
Machine Learning — Introduction, Catégories & Apprentissage par Renforcement
Pr. Boutaina Ettetuani | Université Mundiapolis | 2025-2026

CHAPITRE 1

Introduction au Machine Learning

1. Définition
Le Machine Learning (ML) est une branche de l'IA qui permet à une machine d'apprendre auto
de données, sans règles programmées explicitement, pour faire des prédictions ou aider à l
Formule clé : y = f(x) + bruit
2. Hiérarchie des Concepts
Data Science IA ML Deep Learning. Le Data Mining est complémentaire : il extrait des
de grandes bases de données. Le ML se distingue de la programmation classique : au lieu de
fournit des données et des résultats, et la machine apprend les règles.
3. Statistique vs Machine Learning

Critère	Statistique	Machine Learning
Objectif principal	Expliquer / inférer	Prédire / généraliser
Type de modèles	Simples, interprétables	Parfois complexes (boîte noire)
Librairies clés	statsmodels, R	scikit-learn, PyTorch, TensorFlow
Volume de données	Petits datasets	Grands datasets
Validation	Tests statistiques	Train/test split, cross-validation

Les deux approches sont complémentaires — la statistique explique, le ML prédit.
4. Topologies d'Apprentissage

Topologie	Principe	Sous-types	Exemple
Supervisé	Données + labels fournis	Classification (var. catég.) Régression (var. continue)	Détection spam, prix
Non supervisé	Pas de labels	Clustering, Association Réduction de dimension	Segmentation clients
Semi-supervisé	Peu de labels + beaucoup de données non étiquetées	Self-training, co-training	

5. Types de Données

Données Structurées

Tableaux CSV, SQL, Excel. Variables numériques, catégorielles, ordinales, binaires. Focus principal du cours de ML classique.

Données Non Structurées

Images, texte, audio, vidéo. Requièrent des techniques avancées (Deep Learning, CNN, RNN, Transformers). Volume en forte croissance (80% des données mondiales).

Variables Numériques

Continues (ex: prix, température) ou discrètes (ex: nombre d'enfants). Directement utilisables par les algorithmes ML.

Variables Catégorielles

Nominales (sans ordre: couleur, pays) ou ordinales (avec ordre: faible/moyen/élevé). Nécessitent un encodage avant utilisation.

6. Préparation des Données

Étape	Problème traité	Méthodes courantes
Valeurs manquantes	NaN, données absentes	Suppression, moyenne/médiane, KNN imputation
Valeurs aberrantes	Outliers faussant le modèle	Z-score, IQR, Isolation Forest, visualisation (boxplot)
Normalisation	Variables sur des échelles différentes	Min-Max [0,1], Z-score (μ=0, σ=1)
Encodage	Variables catégorielles non numériques	One-Hot Encoding, Label Encoding, Ordinal Encoding
Unification formats	Dates, majuscules, doublons...	Nettoyage manuel + pandas

Règle d'or : Un bon modèle sur de mauvaises données donnera toujours de mauvais résultats

7. Évaluation du Modèle

Séparation Train/Test

Division des données : 70-80% entraînement, 20-30% test. Jamais utiliser les données de test pendant l'entraînement.

Validation Croisée (k-fold)

Diviser en k parties, entraîner k fois en alternant la partie test. Plus robuste, réduit la variance de l'estimation.

Overfitting

Modèle trop complexe : performe bien sur train, mal sur test. Causes : trop de paramètres, pas assez de données. Solutions : régularisation, dropout.

Underfitting

Modèle trop simple : mauvaise performance sur train ET

Famille	Algorithmes clés	Cas d'usage	Points forts
Clustering	K-Means, DBSCAN, Hierarchical	Segmentation, détection d'anomalies	Sans labels, exploratoire

9. Pipeline ML Complet

Problème → Données → Nettoyage → Feature Engineering → Modèle → Entraînement → Évaluation → Déploiement → Monitoring

1. Définir le problème

Quel est l'objectif ? Classification, régression, clustering ? Quelles sont les métriques de succès (accuracy, RMSE, F1...) ?

2. Collecter & nettoyer

Sources de données, gestion des manquants, outliers, doublons, unification des formats.

3. Feature Engineering

Créer de nouvelles variables pertinentes, encoder les catégorielles, normaliser, sélectionner les features importantes.

4. Modéliser & évaluer

Choisir l'algorithme, entraîner, valider (cross-validation), tuner les hyperparamètres (Grid Search, Random Search).

5. Déployer & monitorer

Intégrer le modèle (API REST, batch, temps réel), surveiller la dérive des données (data drift) et des performances.

Métriques clés

Classification: Accuracy, Precision, Recall, F1, AUC-ROC. Régression: MAE, MSE, RMSE, R^2.

PARTIE 1

Catégories de Machine Learning

1. Cycle de vie d'un projet ML
Un projet ML suit le flux : Sources Prétraitement Modèle ML Prédiction Décision mé
est consacré aux données. Un modèle entraîné modèle utilisable : il faut l'intégrer (API
surveiller et l'améliorer en continu.
Règle clé : Un mauvais dataset produit un mauvais modèle.

2. Types d'apprentissage

Type	Données	Objectif	Algorithmes	Exemple
Supervisé	Étiquetées (X,Y)	Prédire Y	KNN, SVM, Arbres	Prix, spam
Non supervisé	Non étiquetées (X)	Découvrir patterns	K-Means, ACP	Segmentation clients
Semi-supervisé	Partiellement étiq.	Prédire	Combinaison	Détection fraude
Renforcement	Interactions	Maximiser récompense	Q-Learning, PPO	Robot, jeux

3. Apprentissage Supervisé — Détails
Objectif : apprendre une fonction f: X Y telle que f(x) ≈ y, qui généralise sur des do
3.1 Types de problèmes
Régression
Sortie Y continue (ex. prix d'une maison). Modèle : f(x) =
ax + b ou f(x) = wTx + b.
Classification binaire
Y {0,1} (ex. spam / non-spam). Modèle logistique : f(x)
= σ(wTx + b).
Classification multi-classes
Y {1,2,...,K} (ex. reconnaissance de chiffres 0-9).
Classification multi-labels
Y {0,1}K (ex. tags d'images, plusieurs étiquettes
possibles).
3.2 Phases d'entraînement et de test
Entraînement : l'algorithme ajuste ses paramètres θ pour minimiser une fonction de coût. T
de généralisation sur des données jamais vues. Si performance train >> test surapprentis

Algorithme	Type	Principe	Avantages	Limites
KNN	Classif. / Régr.	K voisins les plus proches (vote ou moyenne)	Simple, intuitif	Lent, sensible à K requiert normalisation
Arbre de décision	Classif. / Régr.	Découpe l'espace par questions successives	Très interprétable rapide	Risque overfitting si arbre trop grand
Naive Bayes	Classification	P(C|X) basé sur théorème de Bayes	Rapide, peu de données nécessaires	Hypothèse indépendance souvent irréaliste
SVM	Classif. / Régr.	Hyperplan à marge maximale + kernels	Puissant, bon avec petits datasets	Kernel crucial lourd sur grands sets

5. Fonctions Linéaires vs Non-Linéaires

Critère	Modèle Linéaire	Modèle Non-Linéaire
Frontière	Droite / hyperplan	Courbe complexe
Formule	f(x) = w□x + b	Réseaux, forêts, SVM-kernel
Complexité	Faible	Élevée
Risque	Underfitting	Overfitting

PARTIE 2

Apprentissage par Renforcement

Module 01 — Fondamentaux du RL
1.1 Définition & Positionnement
Le RL est un paradigme où un agent apprend à prendre des décisions en interagissant avec u
maximiser une récompense cumulée. Contrairement au supervisé (labels fournis) et non-super
cachées), le RL n'a pas de dataset fixe : l'agent génère ses propres données en explorant.

Type	Données	Objectif	Feedback
Supervisé	Étiquetées (X, y)	Prédire y	Immédiat (Erreur)
Non Supervisé	Sans étiquettes (X)	Structurer	Aucun
Renforcement	Interactions	Maximiser Gains	Différé (Récompense)

Différence clé : En RL, il n'y a pas de "bonne réponse" fournie. L'agent apprend par essai
1.2 Éléments Clés du RL
S — Espace d'États
Représentation de la situation actuelle. Peut être discret
(ex: Taxi: 500 états) ou continu (ex: position, vitesse). st
 S = état au temps t.
A — Espace d'Actions
Ensemble des décisions possibles. Peut être discret ou
continu. at A = action choisie par l'agent (ex: Haut,
Bas, Gauche, Droite).
R — Fonction de Récompense
Signal de feedback immédiat R(s,a,s') scalaire.
Encode l'objectif. Ex: +20 livraison, -10 erreur, -1 par
step. La récompense différée est un défi majeur.
γ — Facteur de Dépréciation
0 γ < 1 contrôle l'horizon temporel. γ0: myope
(immédiat). γ1: prévoyant (long terme). G = Σ γk rt+k+1
1.3 La Boucle Agent-Environnement
1. L'agent observe l'état st 2. Choisit une action at selon sa politique π 3. L'enviro
L'agent met à jour sa politique 5. Répéter jusqu'à convergence.
Objectif : maximiser G = r1 + γr2 + γ²r3 + ... = Σ γk rt+k+1
Module 02 — Formalisme MDP & Équations de Bellman
2.1 Processus de Décision Markovien (MDP)
Un MDP est un tuple M = (S, A, P, R, γ) où P(s'|s,a) est la probabilité de transition. La
que 'le futur ne dépend que du présent, pas du passé' : P(st+1 | st, at) = P(st+1 | s0,...
2.2 La Politique π
Déterministe
π(s) = a — un état donné mène toujours à la même
action.
Stochastique

2.3 Fonctions de Valeur

V(s) — Valeur d'état : espérance du retour cumulé en partant de s et suivant π. Répond à : en moyenne depuis cet état ?'

$V^{\pi}(s) = E_{\pi}[G_t \mid s_t=s] = E_{\pi}[r_{t+1} + \gamma V^{\pi}(s_{t+1}) \mid s_t=s]$

Q(s,a) — Valeur d'action : valeur d'exécuter l'action a dans l'état s. Plus informative qu choix d'action.

$Q^{\pi}(s,a) = E_{\pi}[G_t \mid s_t=s, a_t=a] = E_{\pi}[r_{t+1} + \gamma Q^{\pi}(s_{t+1},a_{t+1})]$

2.4 Équation de Bellman

Formalisée par Richard Bellman (1957) : 'La valeur d'un état = récompense immédiate + (γ × suivant)'. C'est la récursivité fondamentale du RL. Analogie GPS : temps total = temps jus intersection + temps optimal depuis ce point.

$V^*(s) = \max_a [R(s,a) + \gamma \cdot \Sigma P(s'|s,a) \cdot V^*(s')]$

$Q^*(s,a) = R(s,a) + \gamma \cdot \Sigma P(s'|s,a) \cdot \max_{a'} Q^*(s',a') \Rightarrow \pi^*(s) = \text{argmax}_a Q^*(s,a)$

Module 03 — Algorithmes Tabulaires

3.1 Le Dilemme Exploration vs Exploitation

Exploitation

Utiliser les connaissances actuelles (meilleure action connue). Risque : rester bloqué dans un optimum local.

Exploration

Tester de nouvelles actions pour découvrir de meilleures stratégies. Risque : trop d'essais coûteux sans exploiter les acquis.

Stratégie ε-greedy : avec probabilité ε action aléatoire (explore) ; avec probabilité 1− (exploite). ε-decay : commencer avec ε=1 et diminuer progressivement.

3.2 Q-Learning (Off-Policy)

Algorithme de référence du RL tabulaire. Off-policy : la mise à jour utilise la meilleure suivant (max), indépendamment de l'action réellement choisie. Converge vers la politique o

$Q(s,a) \leftarrow Q(s,a) + \alpha [r + \gamma \cdot \max_{a'} Q(s',a') - Q(s,a)]$

α = taux d'apprentissage (0.01 à 0.5) | δ = r + γ·max Q(s',a') − Q(s,a) = erreur TD (surpr

3.3 SARSA (On-Policy)

On-policy : la mise à jour utilise l'action réellement choisie a' (pas le max). SARSA = St Action. Comportement prudent/conservateur : tient compte de l'exploration dans l'apprentis

$Q(s,a) \leftarrow Q(s,a) + \alpha [r + \gamma \cdot Q(s',a') - Q(s,a)]$

Critère	SARSA (On-Policy)	Q-Learning (Off-Policy)
Mise à jour avec	Action réellement prise a'	Meilleure action max_a' Q
Comportement	Prudent / conservateur	Optimal / agressif
Convergence vers	Politique ε-greedy	Politique optimale Q*
Usage typique	Robotique, sécurité	Jeux, simulateurs
Exemple Cliff Walking	Chemin sûr mais plus long	Chemin court mais risqué

Règle mnémotechnique : SARSA est "peureux" (évite le danger), Q-Learning est "courageux" (global).

Module 04 — Deep RL & Applications

4.1 Deep Q-Network (DQN) — DeepMind 2013

Problème : Atari a 210×160×3 pixels 1018000 états possibles, impossible de stocker une Q remplacer la table par un réseau de neurones Q(s,a) Q_θ(s,a).

Experience Replay

Stocker (s,a,r,s') dans un replay buffer. Tirer des mini-batches aléatoires brise la corrélation temporelle, stabilise l'entraînement.

Réseau Cible (Target Network)

Copie θ mise à jour toutes les N étapes. Évite les oscillations dues à une cible qui bouge. Δθ = τ(θ − θ) [soft update].

Loss DQN : $L(\theta) = E[(r + \gamma \cdot \max_{a'} Q_{\theta^-}(s',a') - Q_\theta(s,a))^2]$

4.2 Policy Gradient & PPO

Optimiser la politique directement sans Q-Table. PPO (Proximal Policy Optimization, OpenAI standard en production. Il évite les mises à jour trop agressives via un clipping du ratio

$L_{CLIP}(\theta) = E[\min(r_t \cdot \hat{A}, \text{clip}(r_t, 1-\varepsilon, 1+\varepsilon) \cdot \hat{A})]$ avec $r_t = \pi_\theta(a|s) / \pi_{\theta_{old}}(a|s)$

Utilisé dans : ChatGPT, robotique, drones, jeux vidéo complexes.

4.3 RLHF — Le Secret de ChatGPT

Reinforcement Learning from Human Feedback aligne les LLMs avec les valeurs humaines. Étap fine-tuning supervisé sur des démonstrations humaines de qualité ; (2) Reward Model — entr sur des préférences humaines comparées ; (3) PPO Fine-Tuning — optimiser le LLM avec PPO p RM(s). GPT-4, Claude, Gemini utilisent tous RLHF ou une variante (DPO, RLAIF).

4.4 Guide de Sélection des Algorithmes

Critère	Q-Learning	SARSA	DQN	PPO
Espace d'états	Fini/petit	Fini/petit	Grand/continu	Grand/continu
Espace d'actions	Discret	Discret	Discret	Discret + Continu
Stabilité	Moyenne	Haute	Moyenne	Haute
Données requises	Peu	Peu	Beaucoup	Beaucoup
Implémentation	Très simple	Très simple	Complexe	Complexe
Exemple typique				

Défi	Description	Solution
Sécurité	Agent non contrôlé peut causer dommages	Constrained RL, safe RL, RLHF

Synthèse Finale — Points Clés à Retenir

ML — Données d'abord

80% du temps est consacré aux données. Un mauvais dataset produit un mauvais modèle. Le prétraitement (normalisation, gestion des manquants, encodage) est une étape clé.

Choisir selon le problème

Supervisé si labels disponibles, non-supervisé pour explorer, renforcement si interaction avec un environnement. Le choix de l'algorithme dépend toujours du problème.

RL = interaction active

L'agent apprend en interagissant — pas de dataset fixe. Il génère ses propres données. Le MDP (S,A,P,R,γ) formalise tout problème de décision séquentielle.

Bellman = récursivité

La valeur d'un état = récompense immédiate + valeur des états futurs (itératif). Q-Learning (off-policy, optimal) vs SARSA (on-policy, prudent).

Deep RL pour les grands espaces

DQN + Experience Replay + Target Network pour les états complexes. PPO pour les actions continues et la production. RLHF pour aligner les LLMs.

Compromis exploration/exploitation

La stratégie ε-greedy est le standard : explorer au début, exploiter en fin d'entraînement. Un agent trop exploiteur reste bloqué dans un optimum local.